

COHORT 3.0

JavaScript Test Solutions

Complete Q&A with Explanations

17

MCQ QUESTIONS

3

CODING PROBLEMS

MCQ QUESTIONS

Q1

MCQ

```
console.log("10" - "5");
```

A) 105

✓ B) 5

C) NaN

D) Error

✓ **Answer: B**

"10" - "5" = 5. The minus operator converts strings to numbers automatically. Only + concatenates strings; all other math operators force number conversion.

✗ **WHY OTHERS ARE WRONG:**

- A) + concatenates strings, but - does not. No concatenation here.
- C) NaN only when conversion fails (e.g., "hello"-5). "10" and "5" are valid numbers.
- D) JS never throws errors for basic arithmetic type coercion.

Q2

MCQ

```
console.log(0 == false);  
console.log(0 === false);
```

A) true true

B) false false

✓ C) true false

D) false true

✓ **Answer: C**

== does loose comparison: false → 0, so 0==false is true. === checks type too: 0 is number, false is boolean → false.

✗ **WHY OTHERS ARE WRONG:**

- A) === never converts types, so 0===false is always false.
- B) == does convert, so 0==false IS true.
- D) 0==false is true, not false.

Q3

MCQ

```
console.log("hello" - 2);
```

A) hello2

B) undefined

✓ C) NaN

D) Error

✓ **Answer: C**

"hello" cannot be converted to a number, so it becomes NaN. NaN - 2 = NaN.

✗ **WHY OTHERS ARE WRONG:**

- A) Only + can concatenate strings. - always tries math.
- B) undefined is a variable value, not an arithmetic result.
- D) JS does not throw errors for NaN arithmetic.

Q4

MCQ

```
let x = 5;  
console.log(x++);  
console.log(++x);
```

✓ A) 5 7

B) 6 7

C) 5 6

D) 6 6

✓ **Answer: A**

x++ returns current value (5) THEN increments (x=6). ++x increments first (x=7) THEN returns (7). So: 5 then 7.

✗ **WHY OTHERS ARE WRONG:**

- B) x++ prints BEFORE incrementing, so first output is 5.
- C) After x++, x=6. ++x makes it 7, not 6.
- D) x++ does increment, so x becomes 6, then ++x makes it 7.

Q5

MCQ

```
console.log(5 ____ "5"); // should be false
```

A) ==

B) =

✓ C) ===

D) !=

✓ Answer: C

5 === "5" is false because types differ (number vs string). === checks both value AND type.

✗ WHY OTHERS ARE WRONG:

- A) 5 == "5" is true (loose comparison converts "5" to 5).
- B) = is assignment, not comparison.
- D) 5 != "5" is false because == sees them as equal, so != gives false.

Q6

MCQ

```
if( ____ ){  
  console.log("Run");  
}  
// Need a falsy value
```

A) []

B) "false"

✓ C) 0

D) {}

✓ Answer: C

0 is falsy in JS. Falsy values: 0, "", null, undefined, NaN, false. The block will NOT run.

✗ WHY OTHERS ARE WRONG:

- A) [] (empty array) is truthy! Arrays are objects and always truthy.
- B) "false" is a non-empty string → truthy.
- D) {} (empty object) is truthy! Objects are always truthy.

Q7

MCQ

```
let day = "Sunday";
switch(day){
  case "Saturday":
  case ____:
    console.log("Weekend");
}
```

A) Sunday

✓ B) "Sunday"

C) sunday

D) "weekend"

✓ **Answer: B**

switch uses === comparison. day = "Sunday" (string). So case must be "Sunday" (string with quotes).

✗ **WHY OTHERS ARE WRONG:**

- A) Sunday without quotes is a variable name, not the string "Sunday".
- C) "sunday" is lowercase — JS is case-sensitive.
- D) "weekend" doesn't match "Sunday" at all.

Q8

MCQ

```
console.log(true + true + false);
```

A) true

B) false

✓ C) 2

D) 0

✓ **Answer: C**

Booleans convert to numbers in math: true=1, false=0. So 1+1+0 = 2.

✗ **WHY OTHERS ARE WRONG:**

- A) true is not returned; numeric addition happens.
- B) false would be 0.
- D) 0 would need false+false+false.

Q9

MCQ

```
console.log(x);  
var x = 10;
```

A) Error

✓ B) undefined

C) 10

D) null

✓ Answer: B

var is hoisted to top of scope but NOT initialized. So x exists but = undefined before assignment.

✗ WHY OTHERS ARE WRONG:

- A) var hoisting prevents ReferenceError. Only let/const throw errors.
- C) x is declared AFTER the log, so value is not 10 yet.
- D) null is an intentional empty value. Hoisting gives undefined.

Q10

MCQ

```
console.log(a);  
let a = 5;
```

A) undefined

B) 5

C) null

✓ D) ReferenceError

✓ Answer: D

let is in Temporal Dead Zone (TDZ) before declaration. Accessing it throws ReferenceError.

✗ WHY OTHERS ARE WRONG:

- A) undefined is for var hoisting. let/const have TDZ.
- B) a has no value yet at that point.
- C) null is intentional empty value, not what hoisting gives.

Q11

MCQ

```
{  
  var x = 50;  
}  
console.log(x);
```

✓ A) 50

B) undefined

C) Error

D) null

✓ **Answer: A**

var ignores block scope {}. It leaks to the function/global scope. So x=50 is accessible outside the block.

✗ **WHY OTHERS ARE WRONG:**

- B) undefined would happen if x was declared but not assigned.
- C) No error — var is global/function scoped.
- D) null is never assigned here.

Q12

MCQ

```
console.log(typeof null);
```

A) null

B) undefined

✓ C) object

D) number

✓ **Answer: C**

This is a famous JS bug! typeof null returns "object" due to a legacy error from the early days of JavaScript.

✗ **WHY OTHERS ARE WRONG:**

- A) "null" is what you'd expect, but JS returns "object".
- B) "undefined" is typeof undefined.
- D) "number" is typeof 42.

Q13

MCQ

Which of these are non-primitive data types?

A) string

✓ B) objects

C) number

✓ D) array

✓ **Answer: B and D**

Objects and Arrays are non-primitive (reference types). Primitives: string, number, boolean, null, undefined, symbol, bigint.

✗ **WHY OTHERS ARE WRONG:**

A) string is a primitive data type.

C) number is a primitive data type.

Q14

MCQ

```
let a;  
let b = null;  
console.log(a == b);
```

A) false

✓ B) true

C) undefined

D) Error

✓ **Answer: B**

undefined == null is true in JS (special rule). They are loosely equal to each other and nothing else.

✗ **WHY OTHERS ARE WRONG:**

A) JS has a special rule: null == undefined is true.

C) undefined is a value, but console.log prints true/false.

D) No error; loose comparison is valid.

Q15

MCQ

```
console.log( ____ ("42")); // should output 42 (number)
```

A) parse

B) String

C) Boolean

✓ D) Number

✓ Answer: D

Number("42") converts the string "42" to the number 42.

✗ WHY OTHERS ARE WRONG:

- A) parse is not a standalone function. parseInt("42") works, but parse() alone doesn't exist.
- B) String() converts TO string, not from string.
- C) Boolean("42") gives true, not 42.

Q16

MCQ

What will be the return value of prompt() by default?

A) number

B) boolean

✓ C) string

D) object

✓ Answer: C

prompt() always returns a string — even if user types "42", you get "42" as string. Use Number() to convert.

✗ WHY OTHERS ARE WRONG:

- A) Never number — always string, even for numeric input.
- B) Never boolean.
- D) Never object.

Q17

MCQ

```
console.log([] + []);
```

A) []

B) 0

✓ C) ""

D) Error

✓ **Answer: C**

[] converts to "" (empty string) when used with +. So "" + "" = "".

✗ **WHY OTHERS ARE WRONG:**

- A) Arrays don't stay as arrays with + operator.
- B) 0 would come from numeric coercion, not string coercion.
- D) No error; JS coerces arrays to strings.

CODING PROBLEMS

Coding 1

Math

100 pts

easy

Check if a Number is a Power of Two

PROBLEM

Given an integer n , return true if n is a power of two (can be written as 2^x where $x \geq 0$), otherwise return false.

INPUT

A single integer n

OUTPUT

Return true if n is a power of two, false otherwise

CONSTRAINT

$-2^{31} \leq n \leq 2^{31} - 1$

💡 SOLUTION

```
class Solution {
  isPowerOfTwo(n) {
    for (let i = 1; i ≤ n; i = i * 2) {
      if (i ≡ n) return true;
    }
    return false;
  }
}
module.exports = Solution;
```

💡 HOW IT WORKS (BEGINNER FRIENDLY)

Start i at 1. Keep multiplying by 2: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \dots$

If i ever equals n exactly $\rightarrow$ it's a power of two!

If i exceeds n without matching $\rightarrow$ not a power of two.

Edge: $n \leq 0 \rightarrow$ loop never runs $\rightarrow$ returns false. ✓

Coding 2

Loop

70 pts

easy

Check if a Number is a Perfect Square

PROBLEM

Check whether a given integer is a Perfect Square. A number is a perfect square if it equals some integer $\times$ itself. Return "Yes" or "No".

INPUT

Integer Number

OUTPUT

Return "Yes" or "No"

CONSTRAINT

Non-negative integer, range 0 to 10^6

💡 SOLUTION

```
class Solution {
  isPerfectSquare(n) {
    for (let i = 0; i * i ≤ n; i++) {
      if (i * i === n) return "Yes";
    }
    return "No";
  }
}
module.exports = Solution;
```

💡 HOW IT WORKS (BEGINNER FRIENDLY)

Try every number i starting from 0.

Check if $i \times i$ equals n .

If yes $\rightarrow$ "Yes"! If $i \times i$ exceeds $n \rightarrow$ "No".

Example: $n=16 \rightarrow 0,1,2,3,4 \rightarrow 4 \times 4=16 \rightarrow$ "Yes"

Example: $n=15 \rightarrow 0,1,2,3,4 \rightarrow 4 \times 4=16 > 15 \rightarrow$ "No"

Check if a Number is Palindromic

PROBLEM

Accept a positive integer and check if it is a palindrome (reads the same forwards and backwards). Return true or false.

INPUT

Integer number

OUTPUT

Return true or false

CONSTRAINT

Positive integer in range 1 to 1,000,000. No negative numbers or non-numeric characters.

💡 SOLUTION

```
class Solution {
  isPalindrome(n) {
    const s = String(n);
    return s === s.split("").reverse().join("");
  }
}
module.exports = Solution;
```

💡 HOW IT WORKS (BEGINNER FRIENDLY)

Convert number to string: 121 → "121"

Split into array: ["1","2","1"]

Reverse: ["1","2","1"]

Join back: "121"

Compare original "121" === "121" → true!

For 123: reverse = "321" → "123" !== "321" → false